FUNCTIONS: DEF & PARAMETERS

A **function** is a named block of code that does one task. Define it once, reuse it anywhere.

1) Define it, then call it

Example :

```
def welcome():
    print("Welcome to Python")
    print("Python is Beginner Friendly")

welcome()
welcome()
```

Output :

```
Welcome to Python
Python is Beginner Friendly
Welcome to Python
Python is Beginner Friendly
```

def only defines the function, the call welcome() is what runs it

2) Parameters and arguments

Example :

```
def greet(name):
    print("Hello", name)

greet("Asha")
greet("Rahul")
```

Output :

```
Hello Asha
Hello Rahul
```

`def greet(name)` name = parameter → `greet("Asha")` "Asha" = argument → name takes the value "Asha"

3) Two parameters, matched by position

Example :

```
def show_student(name, department, course):
    print("Name:", name)
    print("Department:", department)
    print("Course:", course)

show_student("Asha", "Electronics", "Python")
```

Output :

```
Name: Asha
Department: Electronics
Course: Python
```

positional arguments depend on order: swap the values and they land in the wrong places

Quick check

In `def add(a, b)`, `a` and `b` are **parameters**; in `add(10, 5)`, `10` and `5` are **arguments**.

Print Vs Return & Projects

`print()` shows a value on screen. `return` sends it back so the program can reuse it.

1) print() versus return

Example :

```
def add_and_print(a, b):
    print(a + b)

result1 = add_and_print(3, 4)
print("Stored:", result1)
```

Output :

```
7
Stored: None
```

Example :

```
def add_and_return(a, b):
    return a + b

result2 = add_and_return(3, 4)
print("Stored:", result2)
print(result2 * 2)
```

Output :

```
Stored: 7
14
```

2) Project 1: simple calculator

Example :

```
def calculator(a, operator, b):
    if operator == "+":
        return a + b
    elif operator == "-":
        return a - b
    elif operator == "*":
        return a * b
    elif operator == "/":
        if b == 0:
            return "Cannot divide by zero"
        return a / b
    return "Unknown operator"
print(calculator(100, "/", 35))
print(calculator(28, "/", 0))
```

Output :

```
2.857142857142857
Cannot divide by zero
```

3) Project 2: marks calculator

Example :

```
def calculate_average(mark1, mark2, mark3):
    return (mark1 + mark2 + mark3) / 3
def get_result(average):
    if average >= 40:
        return "Pass"
    return "Fail"
average = calculate_average(10, 20, 40)
print("Average:", round(average, 2))
print("Result:", get_result(average))
```

Output :

```
Average: 23.33
Result: Fail
```

Default Args, *args & Data

Give parameters fallback values, accept any number of values, and pass lists or dictionaries.

1) Default arguments

Example :

```
def greet(name, greeting="Hello"):
    print(greeting, name)
greet("Asha")
greet("Asha", "Good morning")
```

Output :

```
Hello Asha
Good morning Asha
```

the default is used only when the caller leaves that argument out

2) *args: any number of positional values

Example :

```
def add_all(*numbers):
    total = 0
    for number in numbers:
        total += number
    return total
print(add_all(10, 20))
print(add_all(5, 10, 15, 20))
```

Output :

```
30
50
```

the star collects all the values into a tuple, the name itself is a habit

Example :

```
def highest_score(*scores):
    if len(scores) == 0:
        return "No scores were provided"
    return max(scores)
print(highest_score(65, 91, 78, 84))
print(highest_score())
```

Output :

```
91
No scores were provided
```

3) Pass a list or a dictionary

Example :

```
def average_marks(marks):
    if not marks:
        return 0
    return sum(marks) / len(marks)
def display_student(student):
    print("Name:", student["name"])
    print("Average:", average_marks(student["marks"]))
display_student({"name": "Rahul", "marks": [75, 64, 82]})
```

Output :

```
Name: Rahul
Average: 73.66666666666667
```

PRACTICE & REVISION

Write each function yourself first, then compare. Each solution is run for real.

Task 1 welcome_student(name) prints Welcome, name.

Solution :

```
def welcome_student(name):
    print("Welcome,", name)

welcome_student("Asha")
```

```
Welcome, Asha
```

Task 2 multiply(a, b) returns the product.

Solution :

```
def multiply(a, b):
    return a * b

print(multiply(4, 5))
```

```
20
```

Task 3 is_pass(marks) returns True or False.

Solution :

```
def is_pass(marks):
    return marks >= 40

print(is_pass(75))
print(is_pass(32))
```

```
True
False
```

Task 4 percentage(obtained, total).

Solution :

```
def percentage(obtained, total):
    if total == 0:
        return "Total cannot be zero"
    return obtained / total * 100

print(percentage(45, 50))
```

```
90.0
```

Task 5 multiply_all(*numbers).

Solution :

```
def multiply_all(*numbers):
    product = 1
    for number in numbers:
        product *= number
    return product

print(multiply_all(2, 3, 4))
```

```
24
```

Task 6 average_marks(*marks), 0 if empty.

Solution :

```
def average_marks(*marks):
    if len(marks) == 0:
        return 0
    return sum(marks) / len(marks)

print(average_marks(70, 80, 90))
print(average_marks())
```

```
80.0
0
```

Argument types in this lesson

Type	How it works	Example
Positional	values are matched by order	`show_student("Asha", "ECE", "Python")`
Default	a fallback is used when omitted	`def greet(name, greeting="Hello")`
*args	any number of values, as a tuple	`def add_all(*numbers)`

Recap

`print()` displays, `return` hands back a value. A function without `return` gives **None**.